Artificial Neural Networks for The Perceptron, Madaline, and Backpropagation Family

Bernard Widrow and Michael A. Lehr

Presented by Zhiyong Cheng
2014-09-08

Outline

- Algorithm development history
- Fundamental Concepts
- Algorithms
  - ➢ Error correction rules
    - ✓ Single Element: linear & nonlinear
    - ✓ Layered Network: nonlinear
  - ➢ Gradient Descent
    - ✓ Single Element: linear & nonlinear
    - ✓ Layered Network: nonlinear

Algorithm Evolution

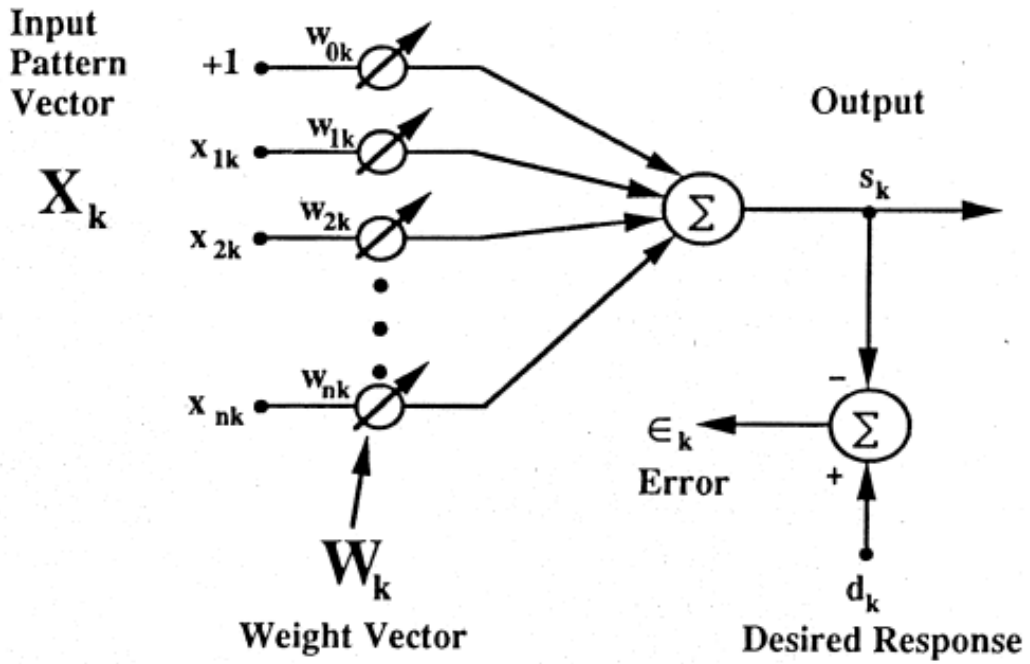
- In 1960, two of the *earliest* and most *important* rules for training adaptive elements were developed: The LMS algorithm (Widrow and his student) and the Perceptron rule (Rosenblatt)
- Facilitate the fast development of neural networks in the early years:
  - Madaline Rule I (MRI) devised by Widrow and his students devised Madaline Rule I (MRI) – earliest popular learning rule for NN with multiple adaptive elements.
  - Application: speech and pattern recognition, weather forecasting, etc.
- Breakthourgh: backpropagation training algorithm
  - Developed by Werbos in 1971 and published in 1974
  - Rediscovered by Parcker in 1982 and reported in 1984, later on Rumelhart, Hinton, and Williams also rediscovered the technique and clearly present their ideas, they finally make it widely known.

Algorithm Evolution

- MRI: hard-limiting quantizers (i.$e.$, $y = sgn(x)$); easy to compute
- Backpropagation network: sigmoid
- MR1 -> MRII (Widrow and Winter) in 1987
- MRII -> MRIII (David Aades in U.S. Naval Weapons Center of China Lake, CA) in 1988 by replacing hard-limiting quantizers with Sigmoid function
  - Widrow and his student first discover that MRIII is equivalent to backpropagation.

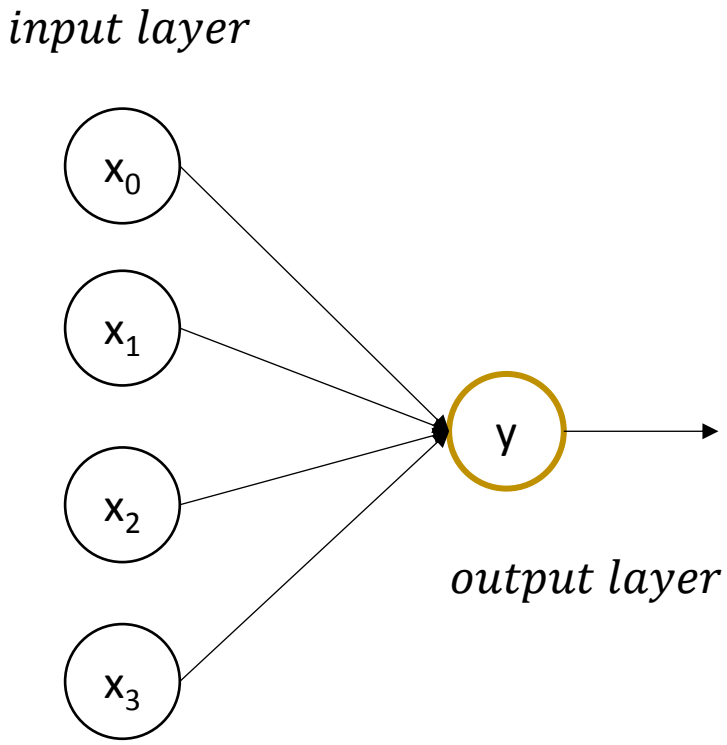
Basic Concepts

- Adaptive linear combiner



$X_k=[x_0, x_{1k}, x_{2k},\ldots, x_{nk}]^T$

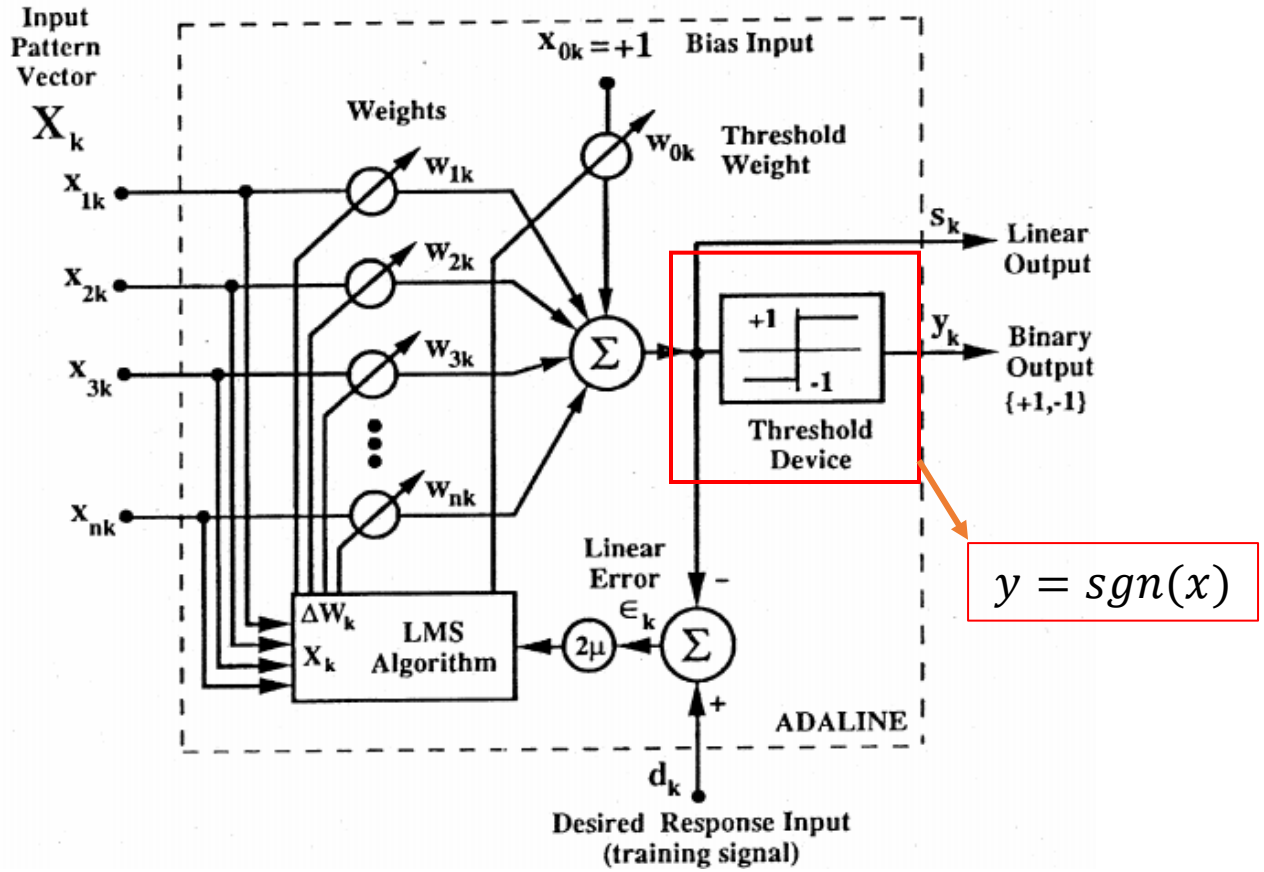
$W_k=[w_{0k}, w_{1k}, w_{2k},\ldots, w_{nk}]^T$ $\quad s_k=X_k^T W_k$

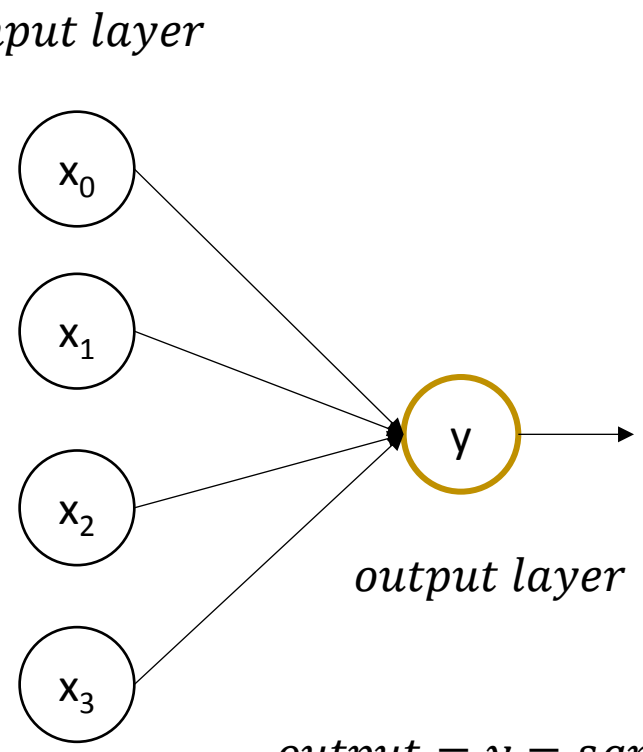


$$y = X_k^T W_k$$

Basic Concepts

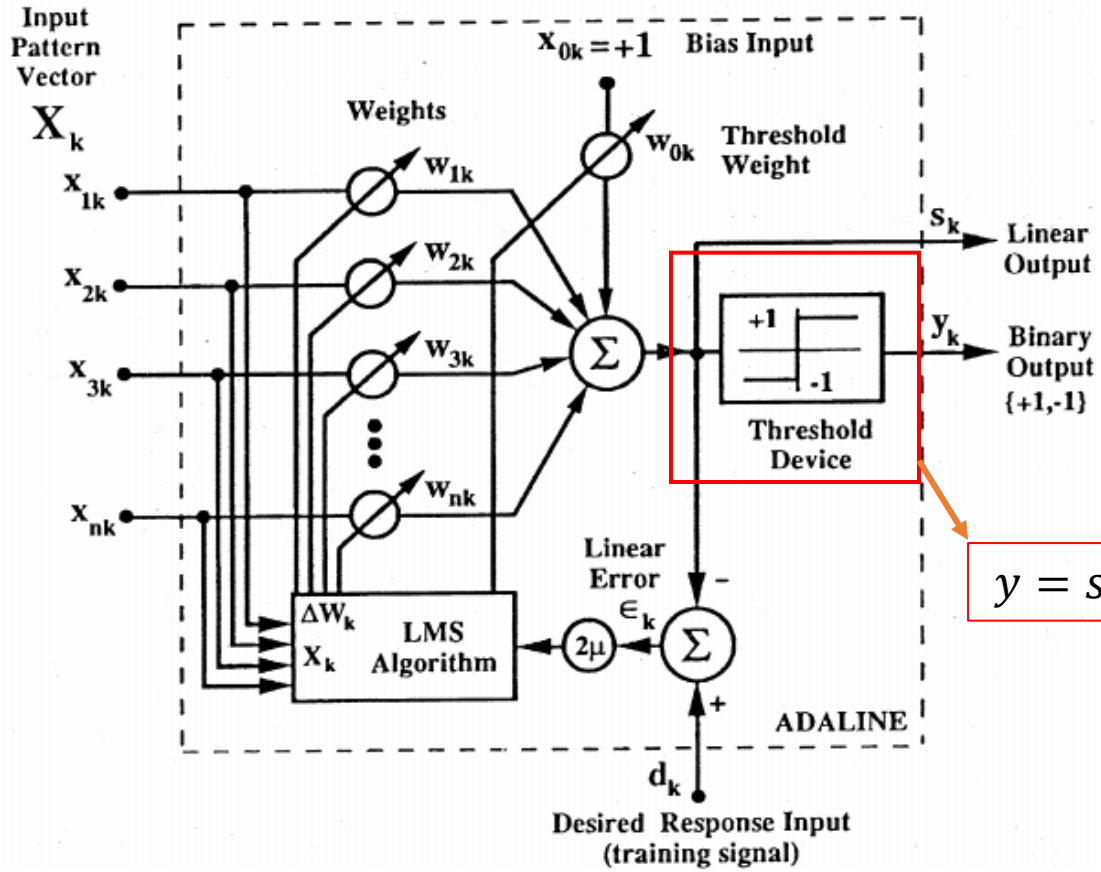
- An adaptive linear element (Adaline) – linear classifier

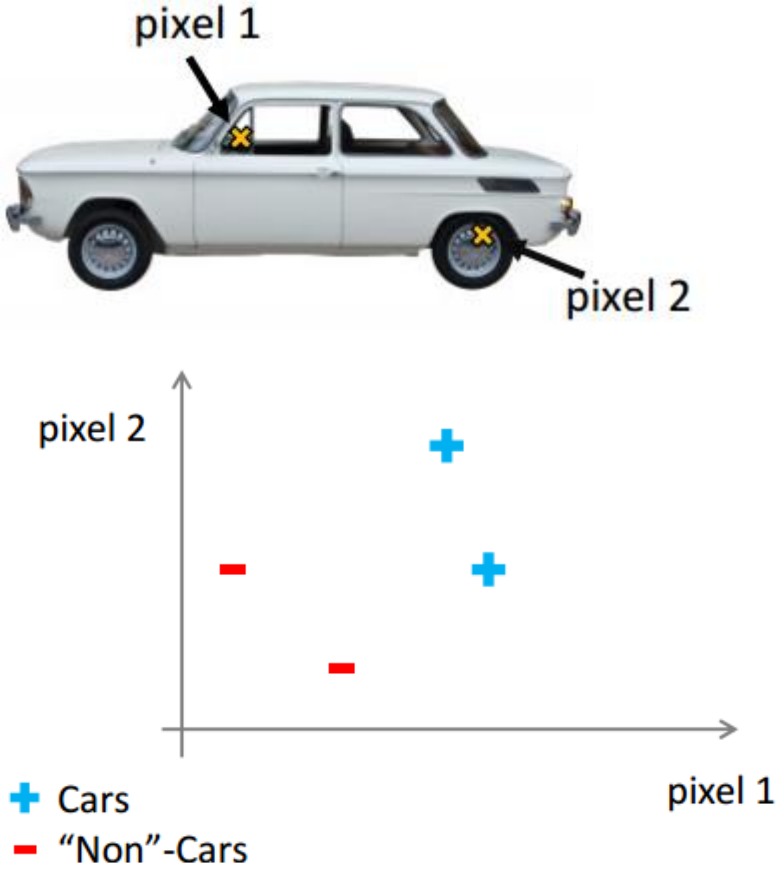




Basic Concepts

- An adaptive linear element (Adaline) – linear classifier

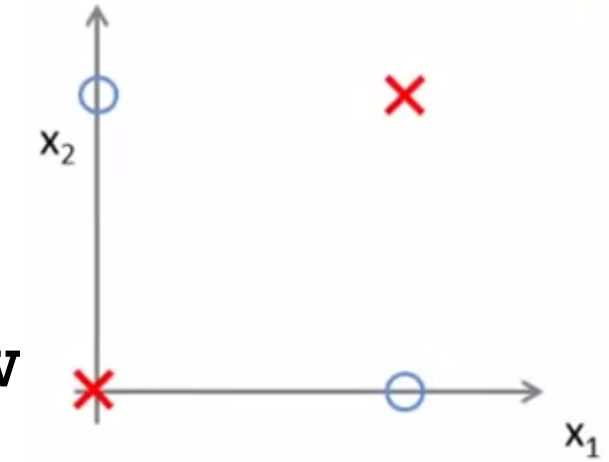


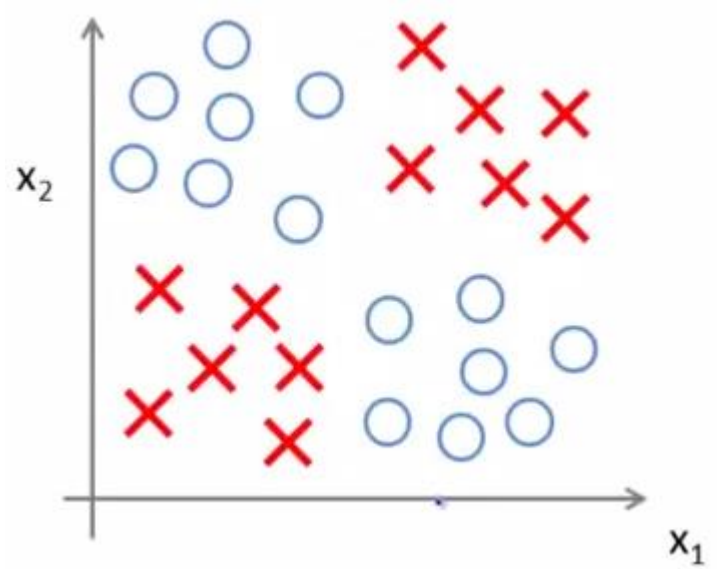


Figures are frim Andrew Ng's online course

Basic Concepts

- Nonlinear classifiers
  - ➢Polynomial discriminant functions
  - ➢Multi-element feedforward neural netw

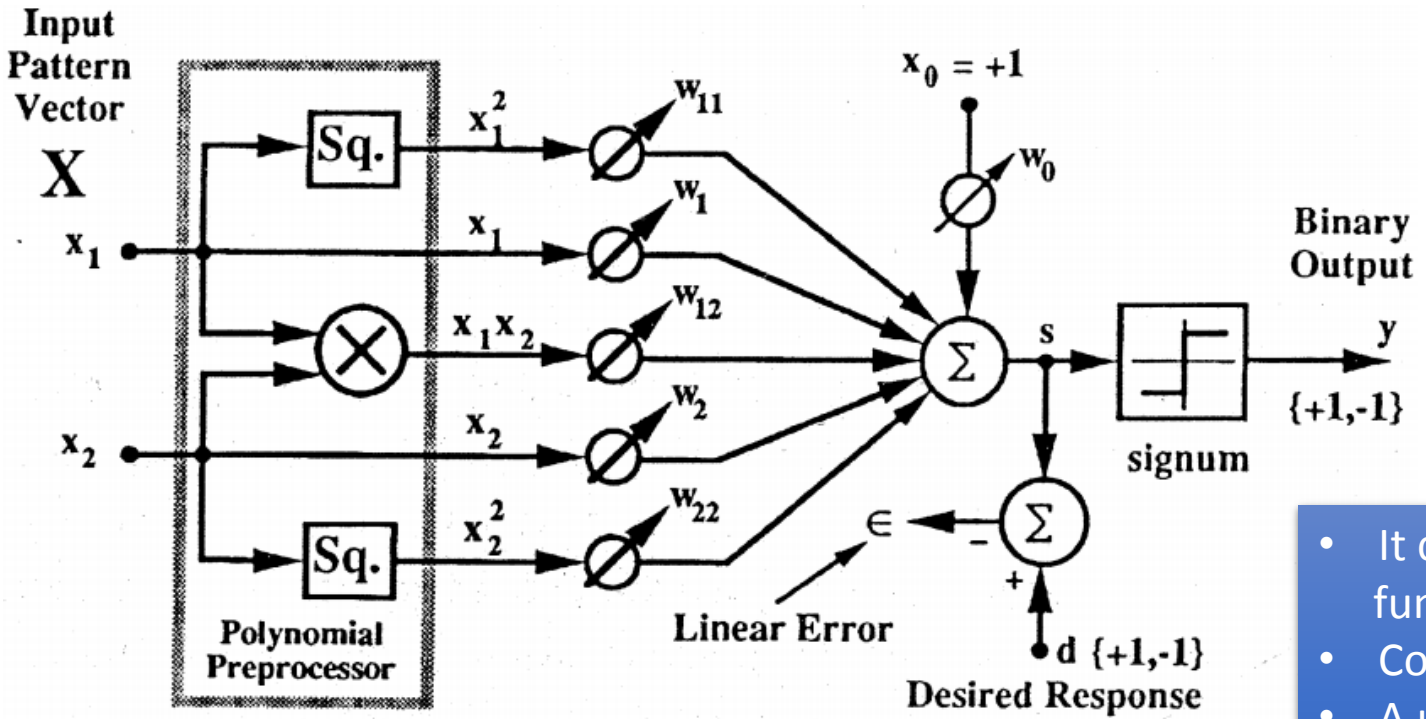




Figures are frim Andrew Ng's online course

Basic Concepts

- Polynomial discriminant function



Initial feature vector:

$$[x_1, x_2]$$

After preprocessor:

$$[x_1^2, x_1, x_1x_2, x_2, x_2^2]$$

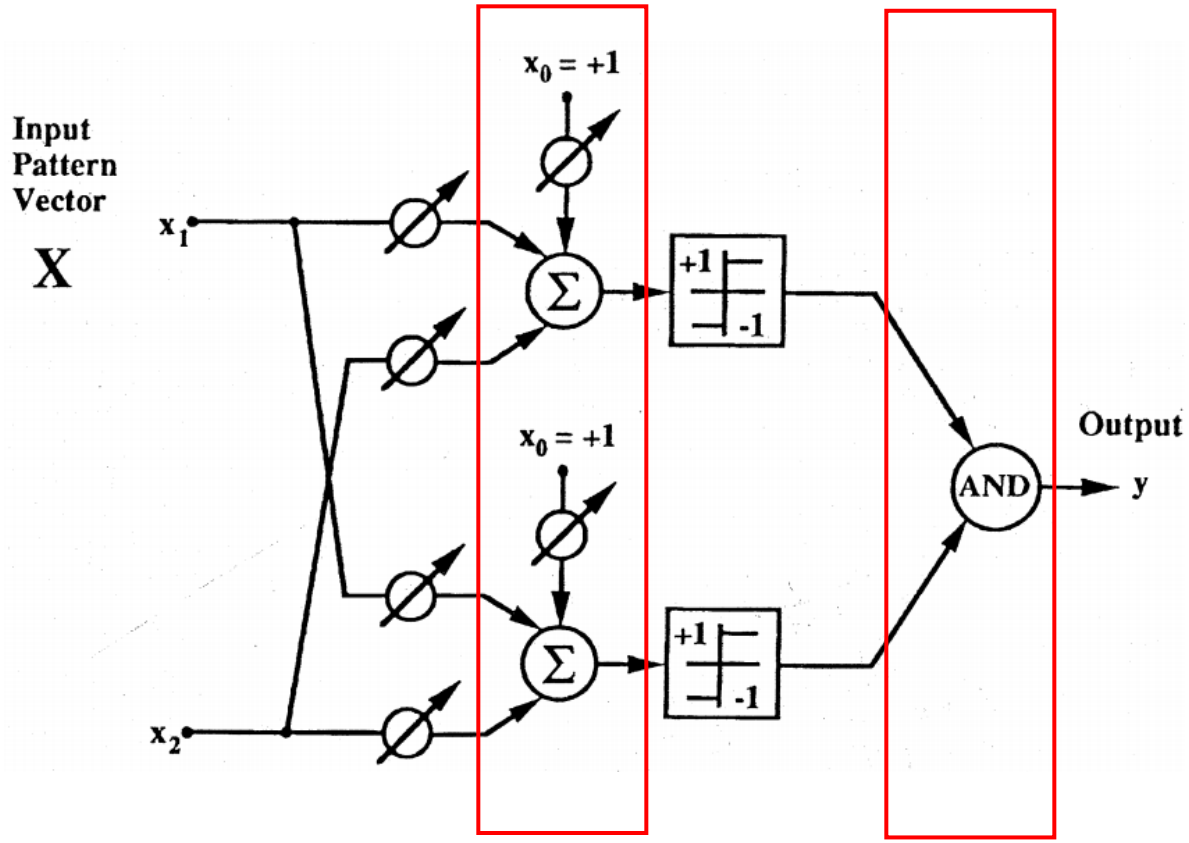
New feature vector

$$[x_1', x_2', x_3', x_4', x_5']$$

- It can provide a variety of nonlinear functions with only a single Adaline element
- Converge faster than layered neural network
- A unique global solution
- However, layered neural networks can obtain better generalization.

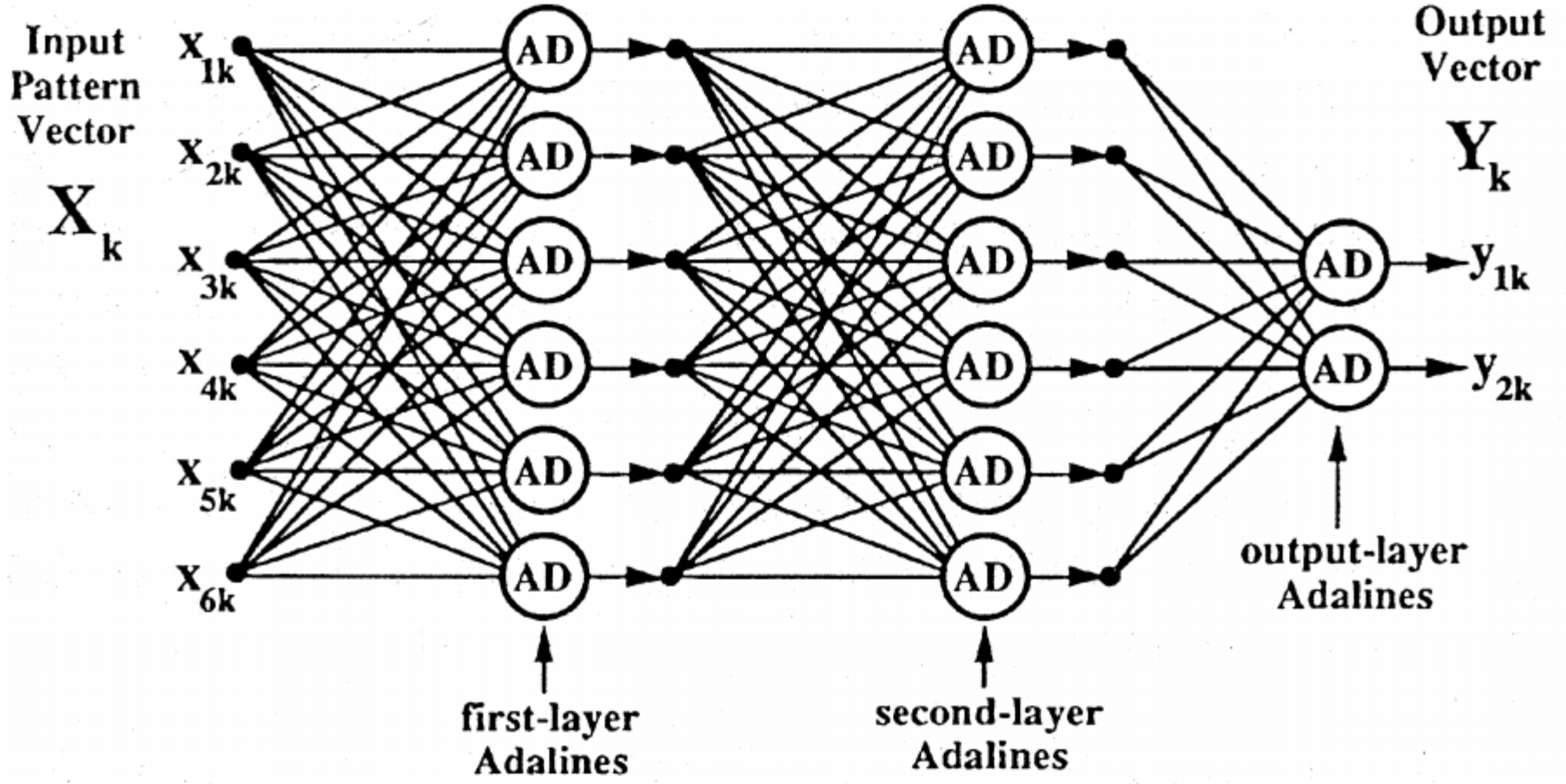
Basic Concepts

- Layered neural networks
- Madaline I
  - Multiple Adaline elements in the first layers
  - Fixed logic devices in the second layer, such as OR, AND, Majority vote etc.



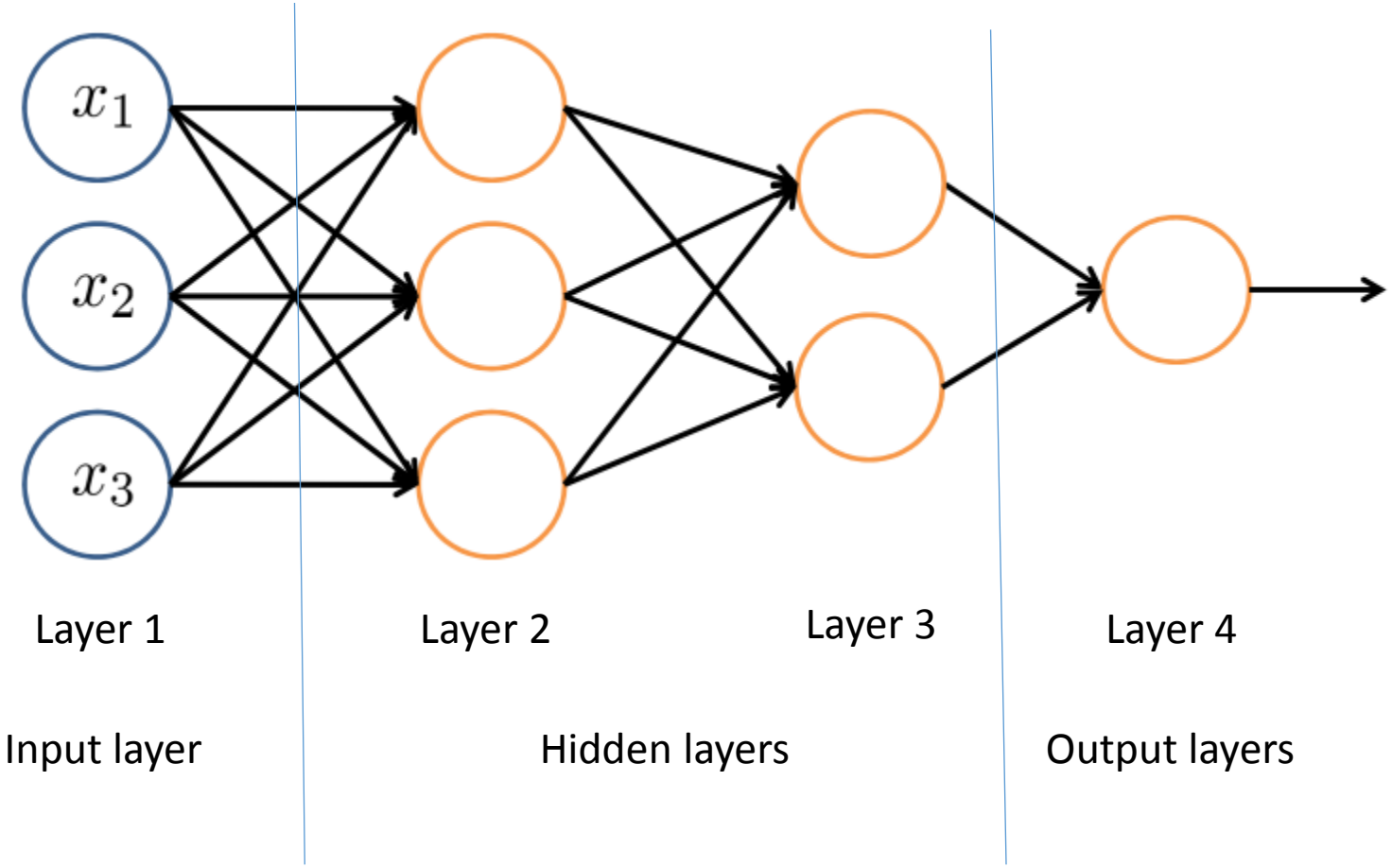
Basic Concepts

- Feedforward Network



Basic Concepts

- Feedforward Network



The Minimal Disturbance Principle

Minimal Disturbance Principle: Adapt to reduce the output error for the current training pattern, with minimal disturbance to responses already learned.

-> lead to the discovery of the LMS algorithm and the Madaline rules.

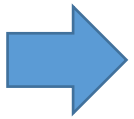
- Single Adaline: Two LMS algorithm, May's rules and Perceptron rule
- Simple Madaline: MRI
- Multi-layer Madalines: MRII, MRIII and Backpropogation algorithm

Adaptive algorithms

- **Error correction rules**: alter the weights to correct a certain proportion of the error in the output response to the present input pattern,
  - ➢ α-LMS algorithm

Weight alteration:

$$W_{k+1} = W_k + \alpha \frac{\varepsilon_k X_k}{|X_k|^2}$$



Error correction:

$$\Delta\varepsilon_k = -\alpha\varepsilon_k$$

Adaptive algorithms

- **Gradient rules**: alter the weights during each pattern presentation by gradient descent with the objective of reducing mean-square-error, average over all training patterns,
  - ➢ μ-LMS algorithm

Weight alteration:

$$\mathbf{W}_{k+1} = \mathbf{W}_k + 2\mu\epsilon_k\mathbf{X}_k$$

Error correction:

$$\nabla_k \triangleq \left\{ \begin{array}{c} \frac{\partial E[\epsilon_k^2]}{\partial w_{0k}} \\ \vdots \\ \frac{\partial E[\epsilon_k^2]}{\partial w_{nk}} \end{array} \right\}_{\mathbf{W}=\mathbf{W}_k}$$

Square error

Error Correction Rules: Adaline
Linear rule: α-LMS Algorithm

- Weight updated rule:

$$W_{k+1} = W_k + \alpha \frac{\varepsilon_k X_k}{|X_k|^2}$$

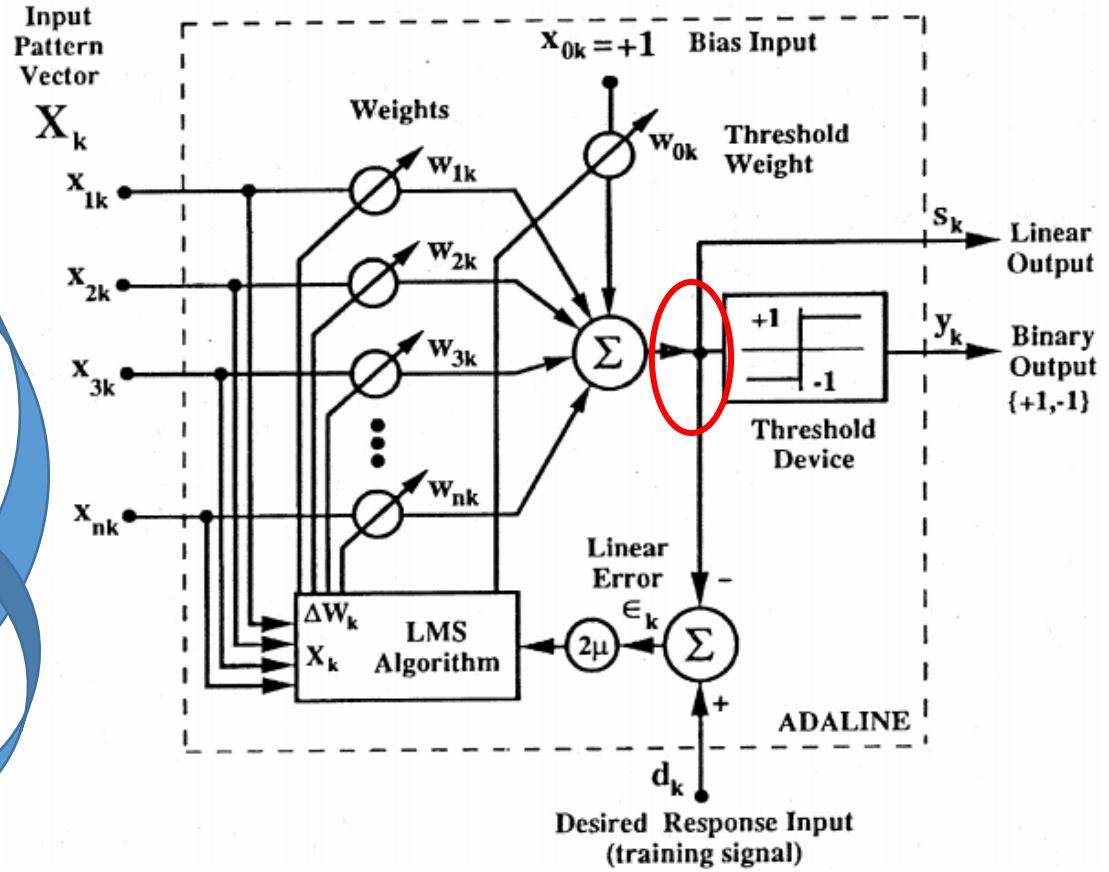
- Error: $\varepsilon_k = d_k - W_k^T X_k$
- Weight change yields an error change

$$\Delta\varepsilon_k = \Delta(d_k - W_k^T X_k) = -X_k^T \Delta W_k$$

$$\Delta W_k = W_{k+1} - W_k = \alpha \frac{\varepsilon_k X_k}{|X_k|^2}$$

$$\Delta\varepsilon_k = -\alpha \frac{\varepsilon_k X_k^T X_k}{|X_k|^2} = -\alpha \Delta\varepsilon_k$$

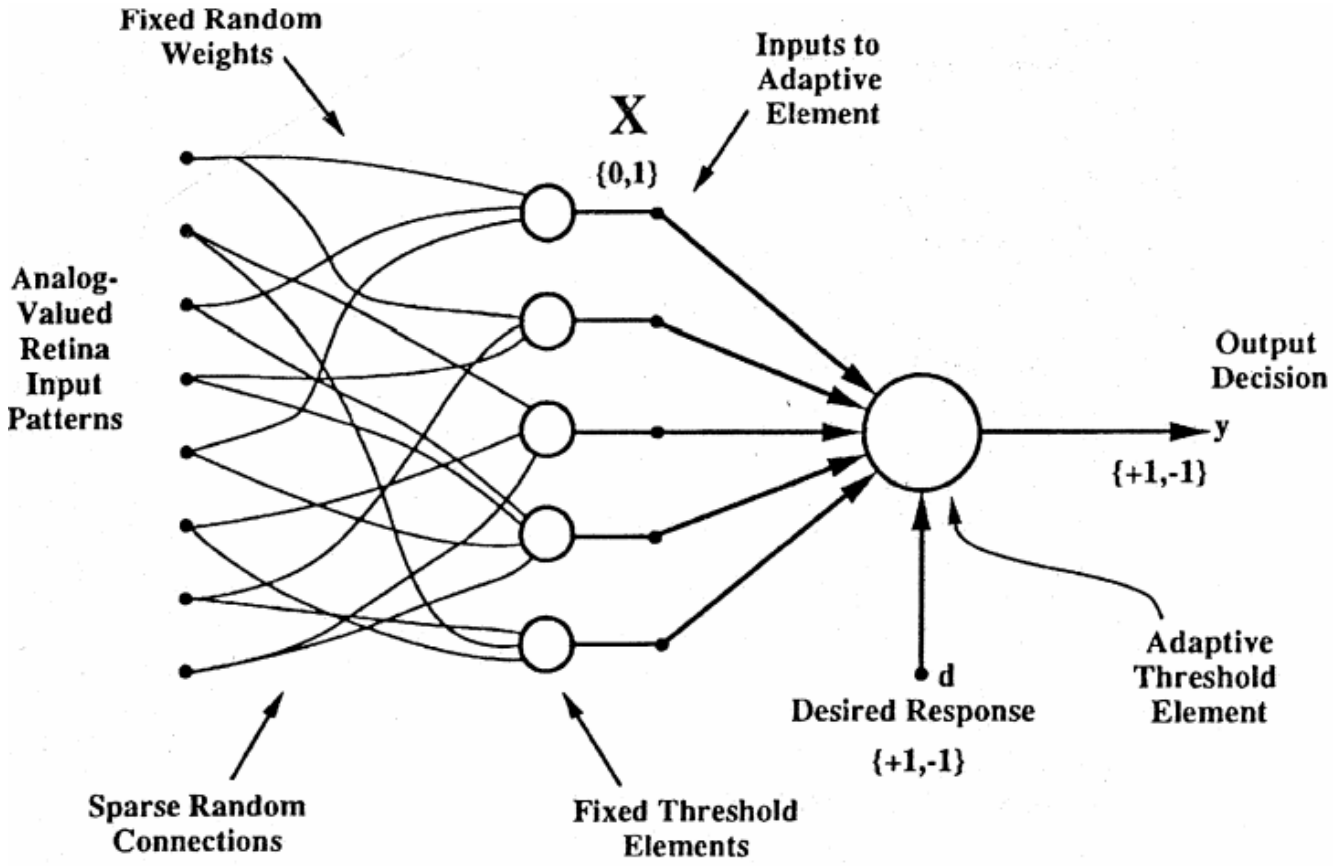
- Learning rate: $0.1 < \alpha < 1$



A Single Adaline

Error Correction Rules: Adaline
Nonlinear: The perceptron Learning rule

- α-Perceptron (Rosenblatt)



Error Correction Rules: Adaline Nonlinear: The perceptron Learning rule

- Quantizer error:

$$\tilde{\varepsilon} = d_k - y_k$$

- Weight update:

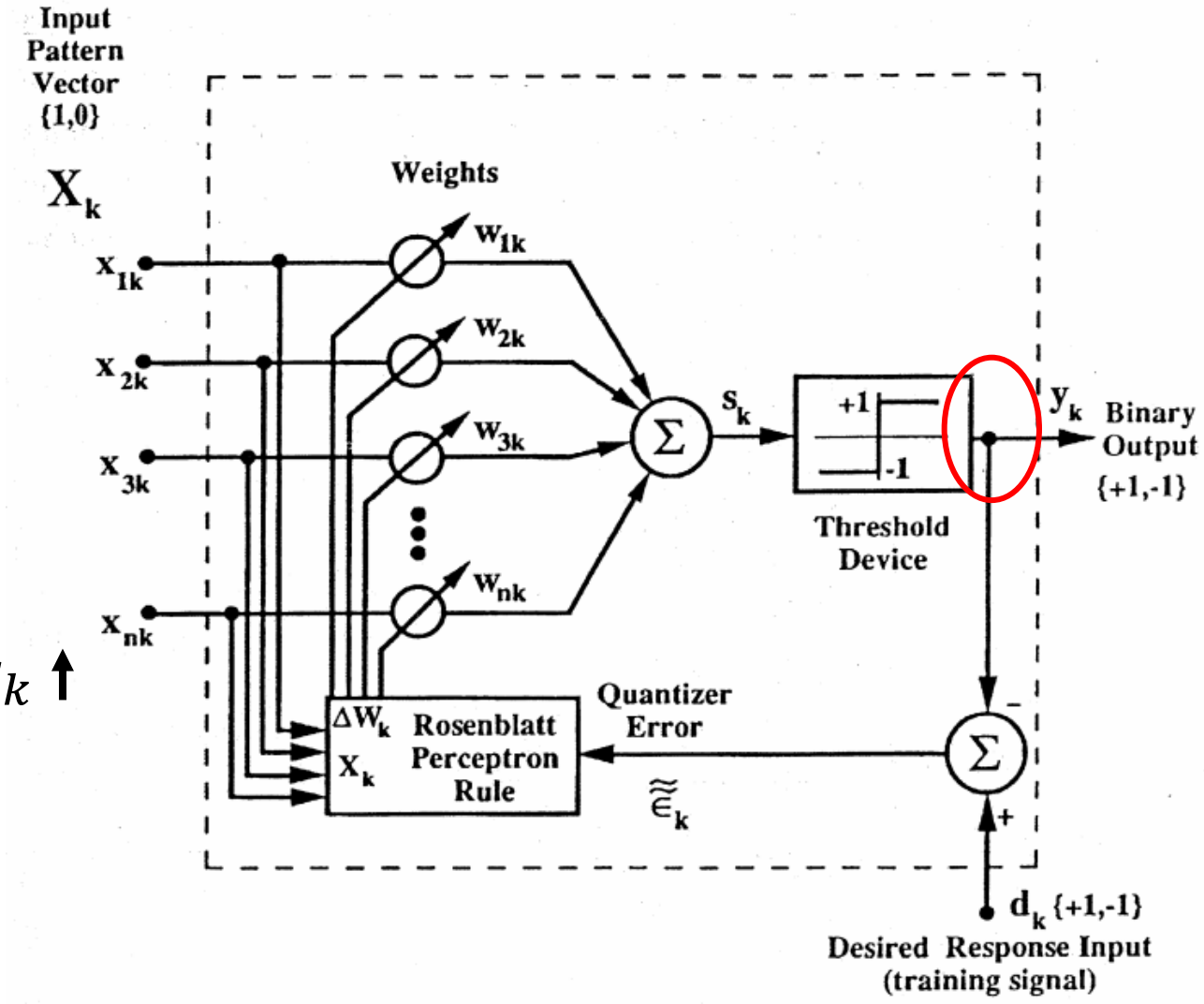
$$W_{k+1} = W_k + \alpha \frac{\tilde{\varepsilon}}{2} X_k$$

$$\alpha = 1$$

$\tilde{\varepsilon} > 0$ -> $X_k = 1, W_{k+1} = W_k + \alpha \frac{|\tilde{\varepsilon}|}{2}$;

$\tilde{\varepsilon} > 0$ -> $d_k = 1, y_k = -1;$ -> s_k ↑

$\tilde{\varepsilon} < 0$ -> $X_k = 1, W_{k+1} = W_k - \alpha \frac{|\tilde{\varepsilon}|}{2}$



Error Correction Rules: Adaline
α-LMS vs. Perceptron Rule

- Value α
  - α-LMS – controls stability and speed of convergence
  - Perceptron rule – does not affect the stability of the perceptron algorithm, and it affects convergence time only if the initial weight vector is nonzero
- α-LMS – both binary and continues responses,

  Perceptron rule – only with binary desired responses.

Error Correction Rules: Adaline
α-LMS vs. Perceptron Rule

- Linearly separable training patterns
  - Perceptron rule – separate any linearly separable set
  - α-LMS - may fail to separate linearly separable set
- Nonlinearly separable training patterns
  - Perceptron rule–goes on forever is not linearly separable, and often does not yield a low-error solution.
  - α-LMS - does not lead to unreasonable weight solution

Error Correction Rules: Adaline Nonlinear: May's Algorithms

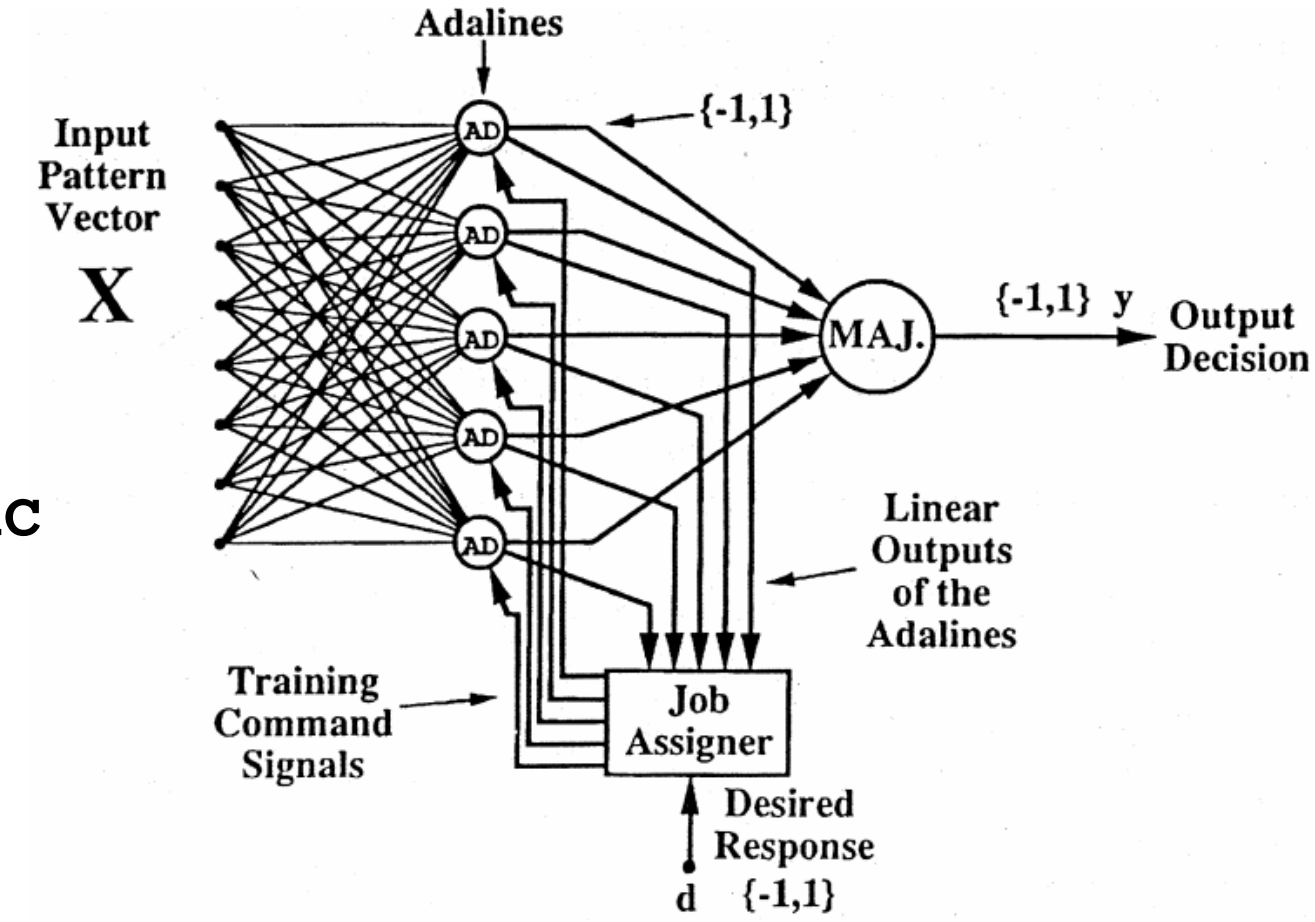
- May's Algorithms

$$\mathbf{W}_{k+1} = \begin{cases} \mathbf{W}_k + \alpha \tilde{\tilde{\epsilon}}_k \frac{\mathbf{X}_k}{2|\mathbf{X}_k|^2} & \text{if } |s_k| \geq \gamma \\ \mathbf{W}_k + \alpha d_k \frac{\mathbf{X}_k}{|\mathbf{X}_k|^2} & \text{if } |s_k| < \gamma \end{cases}$$

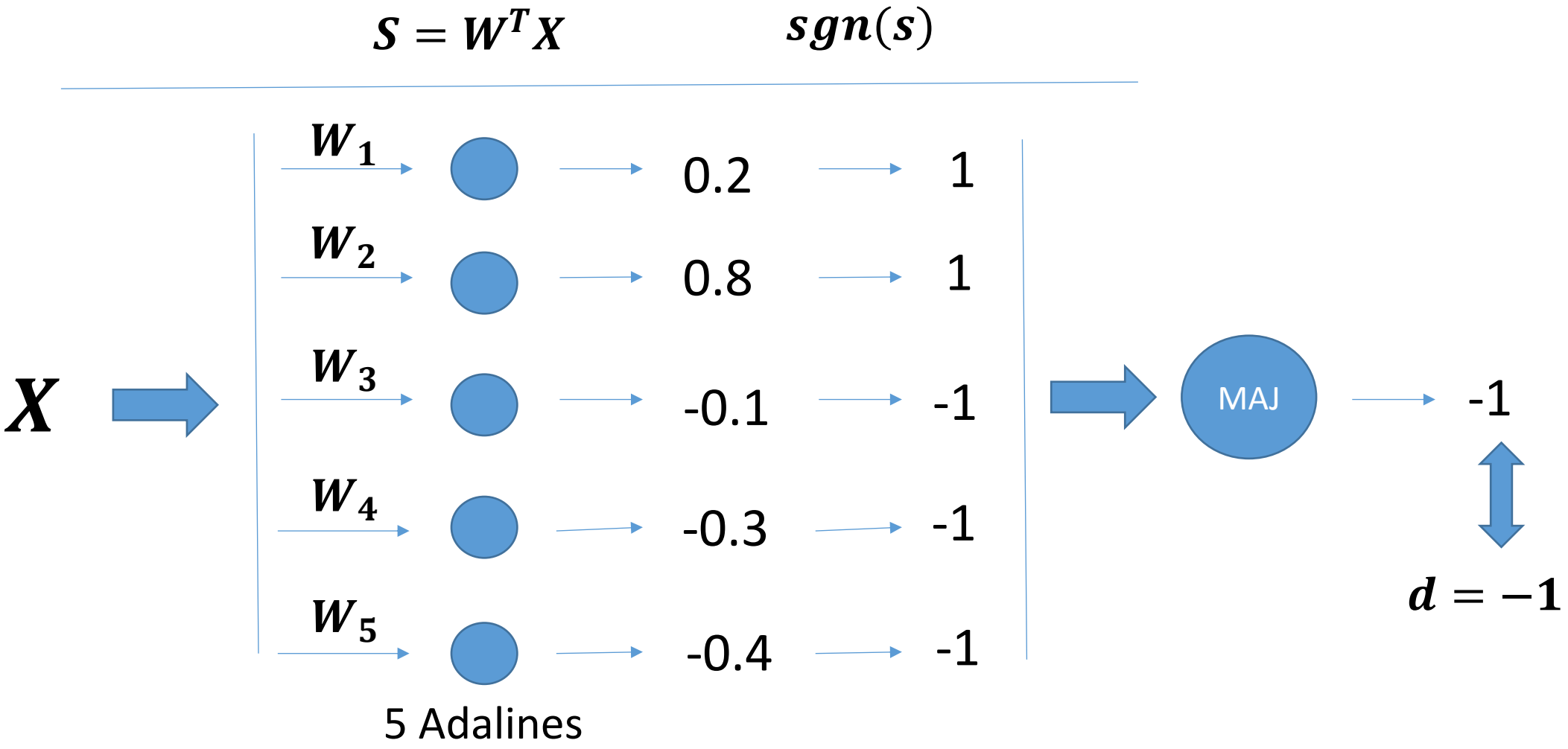
- Separate any linearly separable set
- For nonlinearly separable set, May's rule performs much better than Perceptron rule because a sufficiently large dead zone tends to cause the weight vector to adapt away from zero when any reasonably good solution exists

Error Correction Rules: Madaline
Madaline Rule I (MRI)

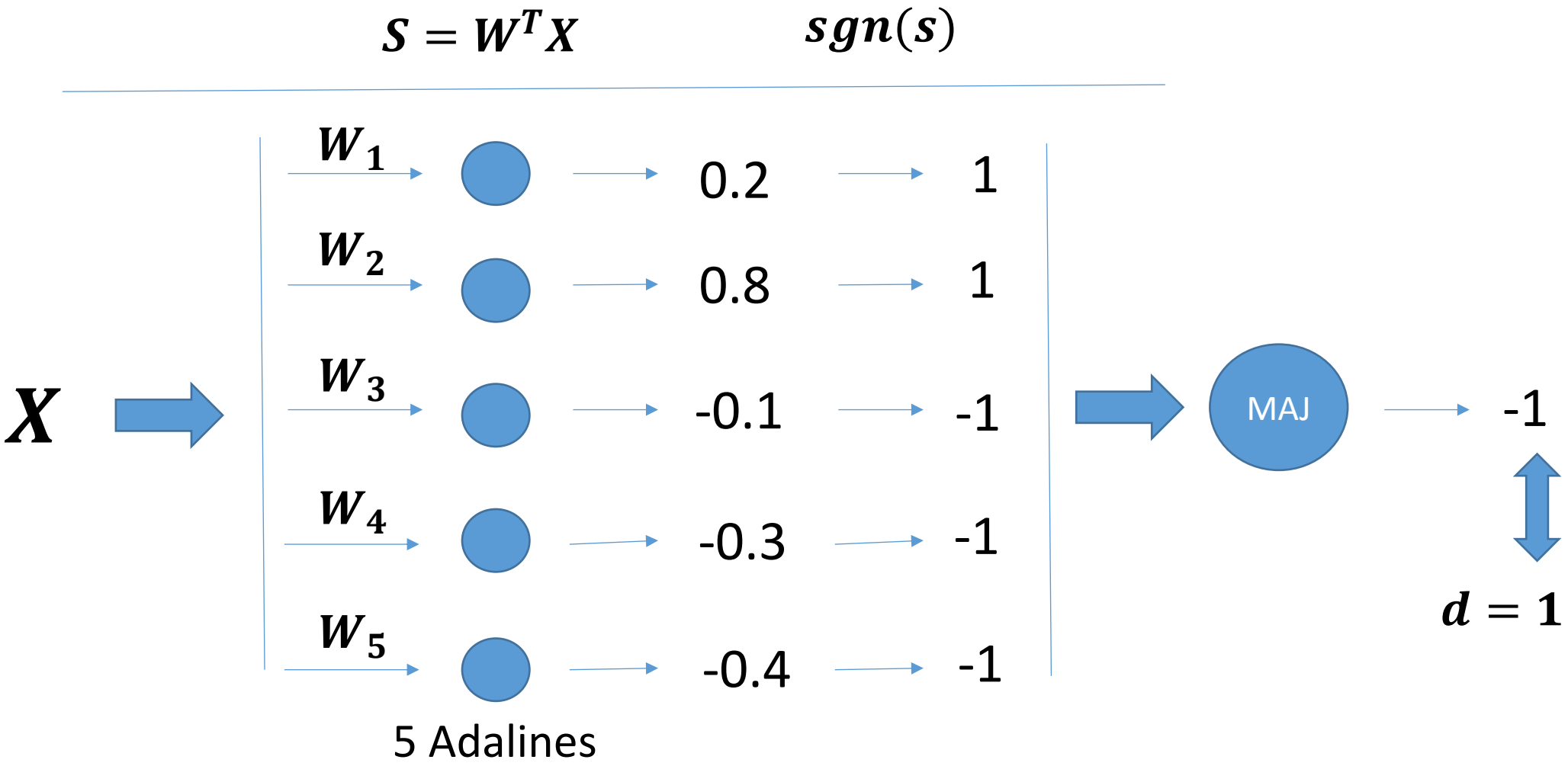
- First layer:
  - hard-limited Adaline elements
- Second layer:
  - a single fixed threshold logic element



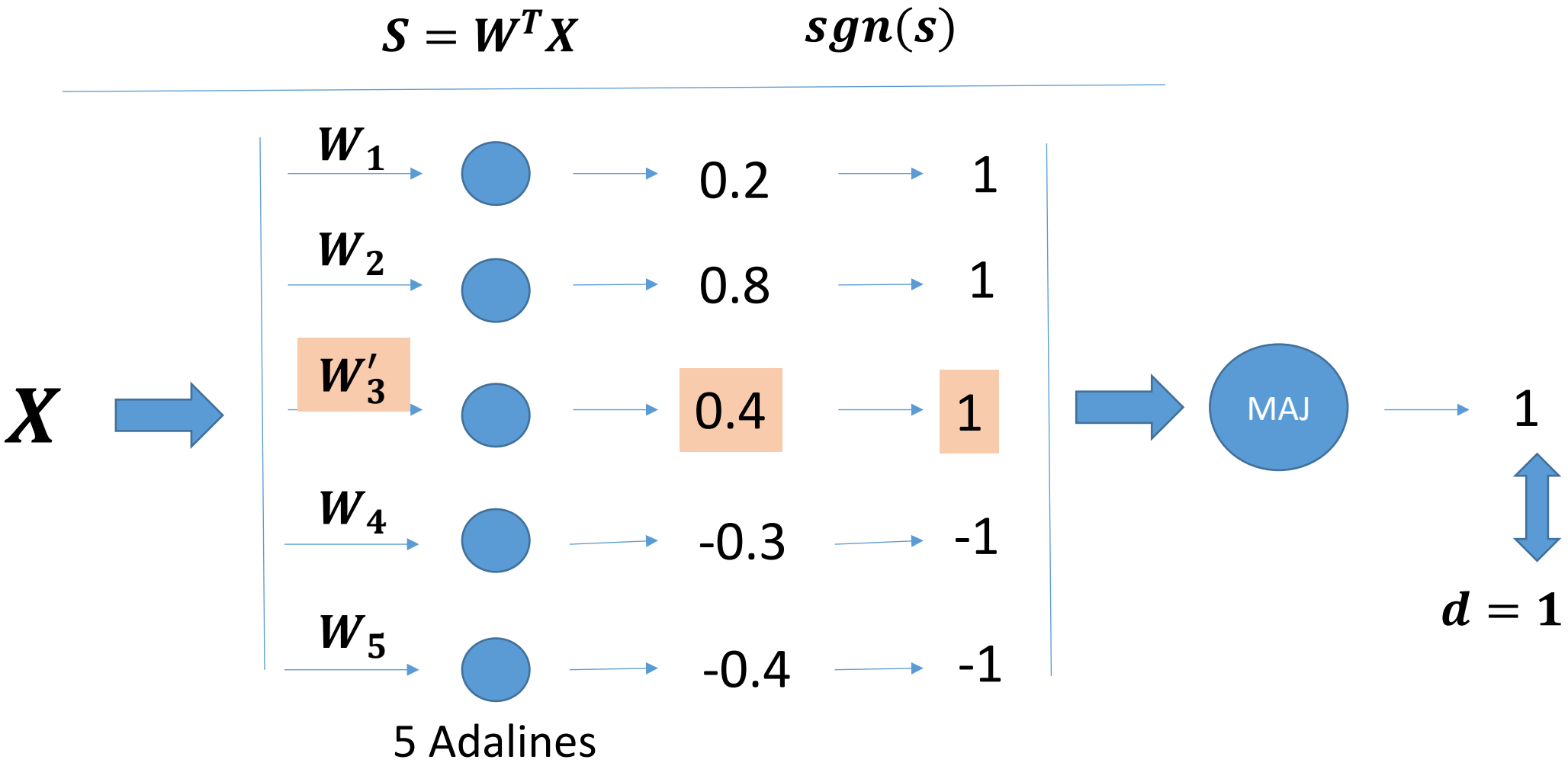
Error Correction Rules: Madaline
Madaline Rule I (MRI)



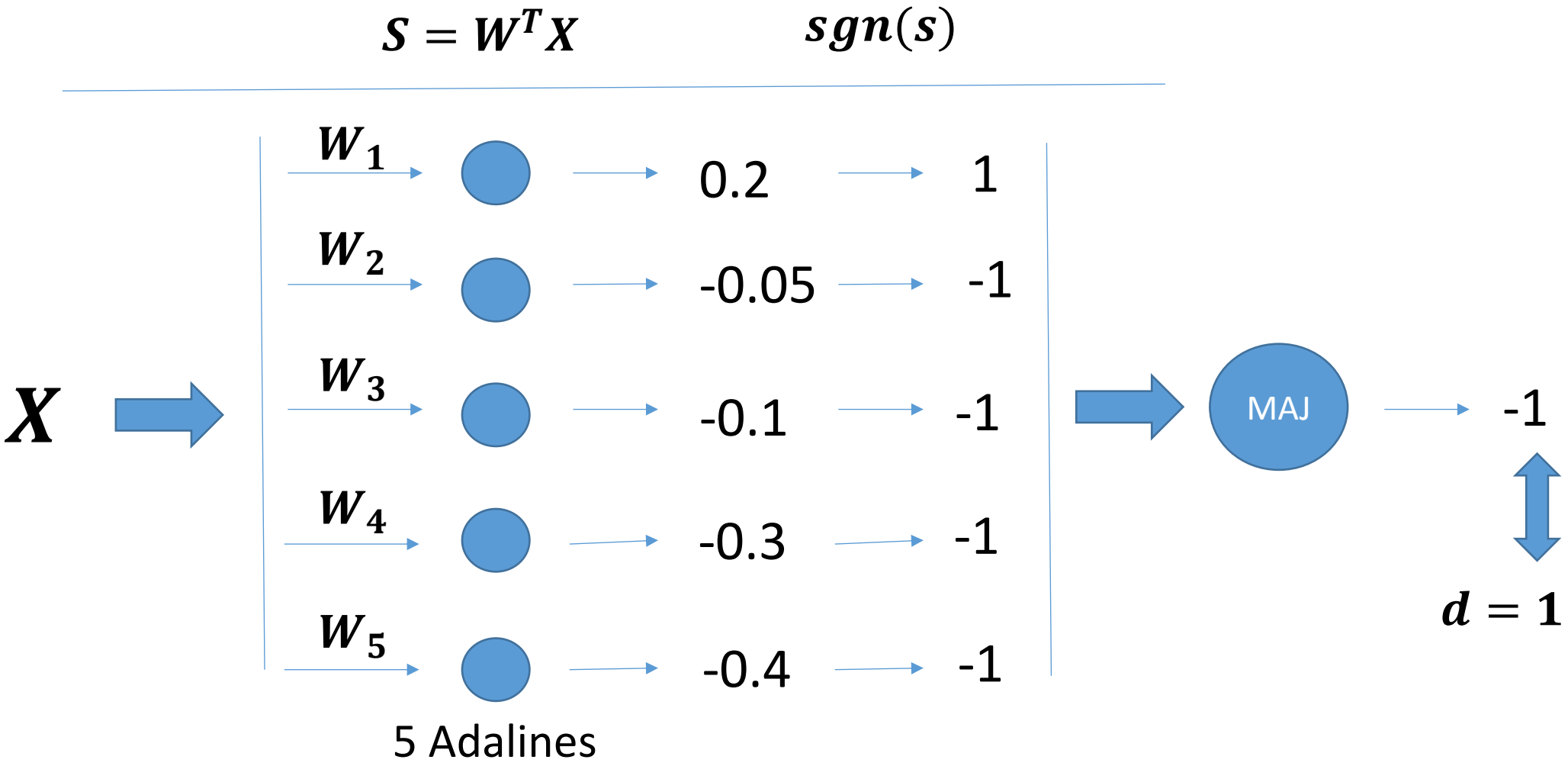
Error Correction Rules: Madaline
Madaline Rule I (MRI)



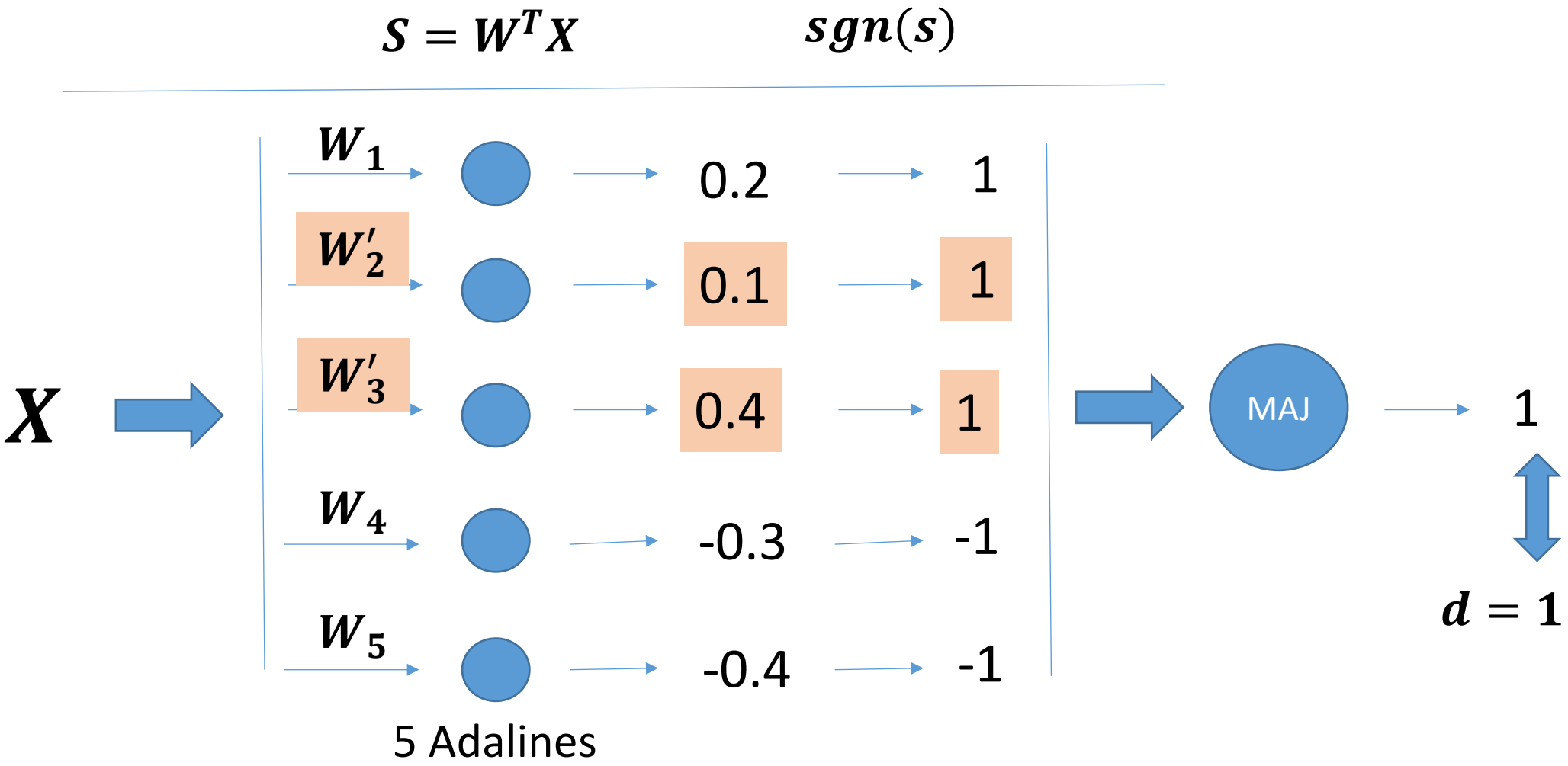
Error Correction Rules: Madaline
Madaline Rule I (MRI)



Error Correction Rules: Madaline
Madaline Rule I (MRI)



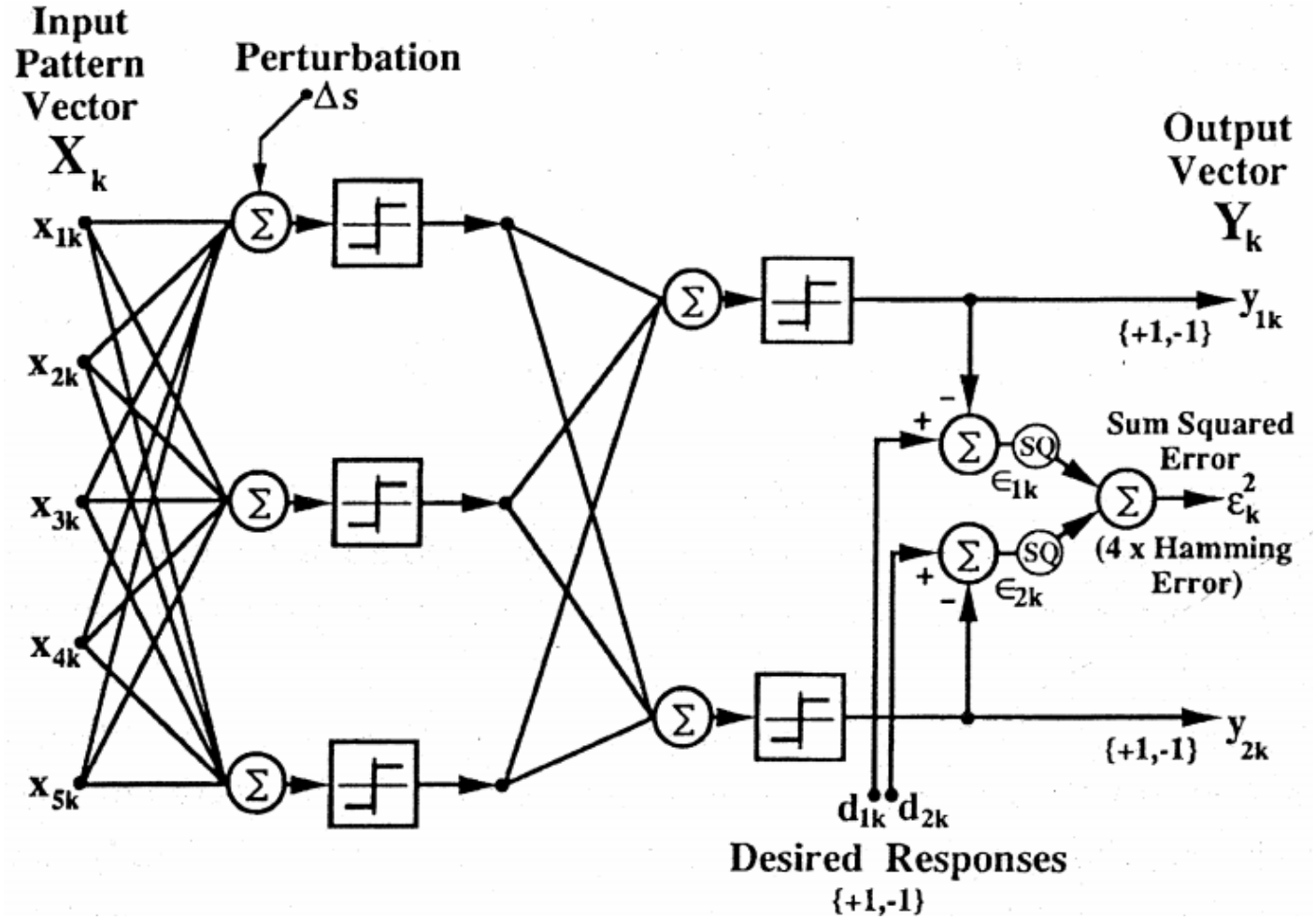
Error Correction Rules: Madaline
Madaline Rule I (MRI)



Error Correction Rules: Madaline Madaline Rule I (MRI)

- Weights are initially set to small random values
- Weight vector update: can be changed aggressively using absolute correction *or* can be adapted by the small increment determined by the α-LMS algorithm
- Principle: assign responsibility to the Adaline or Adalines that can most easily assume it
- Pattern presentation sequence should be random

Error Correction Rules: Madaline Madaline Rule II (MRII)



Typical two-layer Madaline II architecture

Error Correction Rules: Madaline Madaline Rule II (MRII)

- Weights are initially set to small random values
- Random pattern presentation sequence
- Adapting the first-layer Adalines
  1. Select the smallest linear output magnitude
  2. Perform a "trial adaption" by adding a perturbation Δs of suitable amplitude to invert its binary output
  3. If the output error is reduced, remove Δs, and change the weight of the select Adaline using α-LMS algorithm
  4. Perturb and update oter Adalines in the first layer with "sufficiently small" s_k output

     - can reversing pairs, triples, etc
  5. After exhausting possibilities with the first layer, move on to next layer and proceed in a like manner
  6. Random select a new training pattern and repeat the procedure.

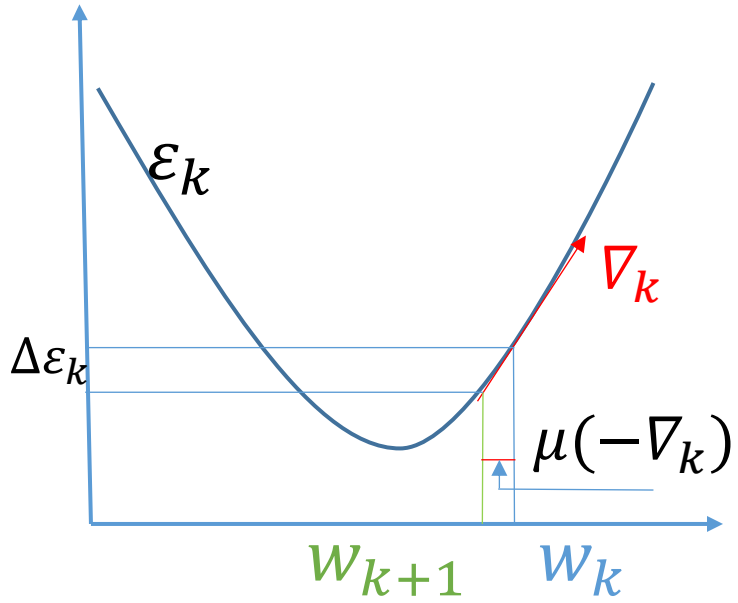
Steepest-Descent

- **Steep-descen**t: measure the gradient of the mean-square-error function and alter the weight vector in the negative direction of the measured gradient.

$$\boldsymbol{W}_{k+1} = \boldsymbol{W}_k + \mu(-\nabla_k)$$

- μ: controls stability and rate of convergence

- ∇_k: the value of the gradient at a point on the MSE surface corresponding to $W = W_k$



Steepest-Descent – Single Element μ-LMS algorithm

- Instantaneous linear error

$$\hat{\nabla}_k = \frac{\partial \epsilon_k^2}{\partial \mathbf{W}_k} = \begin{Bmatrix} \frac{\partial \epsilon_k^2}{\partial w_{0k}} \\ \vdots \\ \frac{\epsilon_k^2}{\partial w_{nk}} \end{Bmatrix} \quad \textit{instead of} \quad \nabla_k \triangleq \begin{Bmatrix} \frac{\partial E[\epsilon_k^2]}{\partial w_{0k}} \\ \vdots \\ \frac{\partial E[\epsilon_k^2]}{\partial w_{nk}} \end{Bmatrix}_{\mathbf{W}=\mathbf{W}_k}$$

Steepest-Descent – Single Element Linear rules: μ-LMS algorithm

- Weight update

$$\mathbf{W}_{k+1} = \mathbf{W}_k + \mu\left(-\hat{\nabla}_k\right) = \mathbf{W}_k - \mu\frac{\partial \epsilon_k^2}{\partial \mathbf{W}_k}$$

$$\mathbf{W}_{k+1} = \mathbf{W}_k - 2\mu\epsilon_k\frac{\partial \epsilon_k}{\partial \mathbf{W}_k}$$

$$= \mathbf{W}_k - 2\mu\epsilon_k\frac{\partial\left(d_k - \mathbf{W}_k^T\mathbf{X}_k\right)}{\partial \mathbf{W}_k}$$

d_k and X_k are independent of W_k

$$\mathbf{W}_{k+1} = \mathbf{W}_k + 2\mu\epsilon_k\mathbf{X}_k$$

- μ: controls stability and rate of convergence; $0 < \mu < \frac{1}{trace[\mathbf{R}]}$

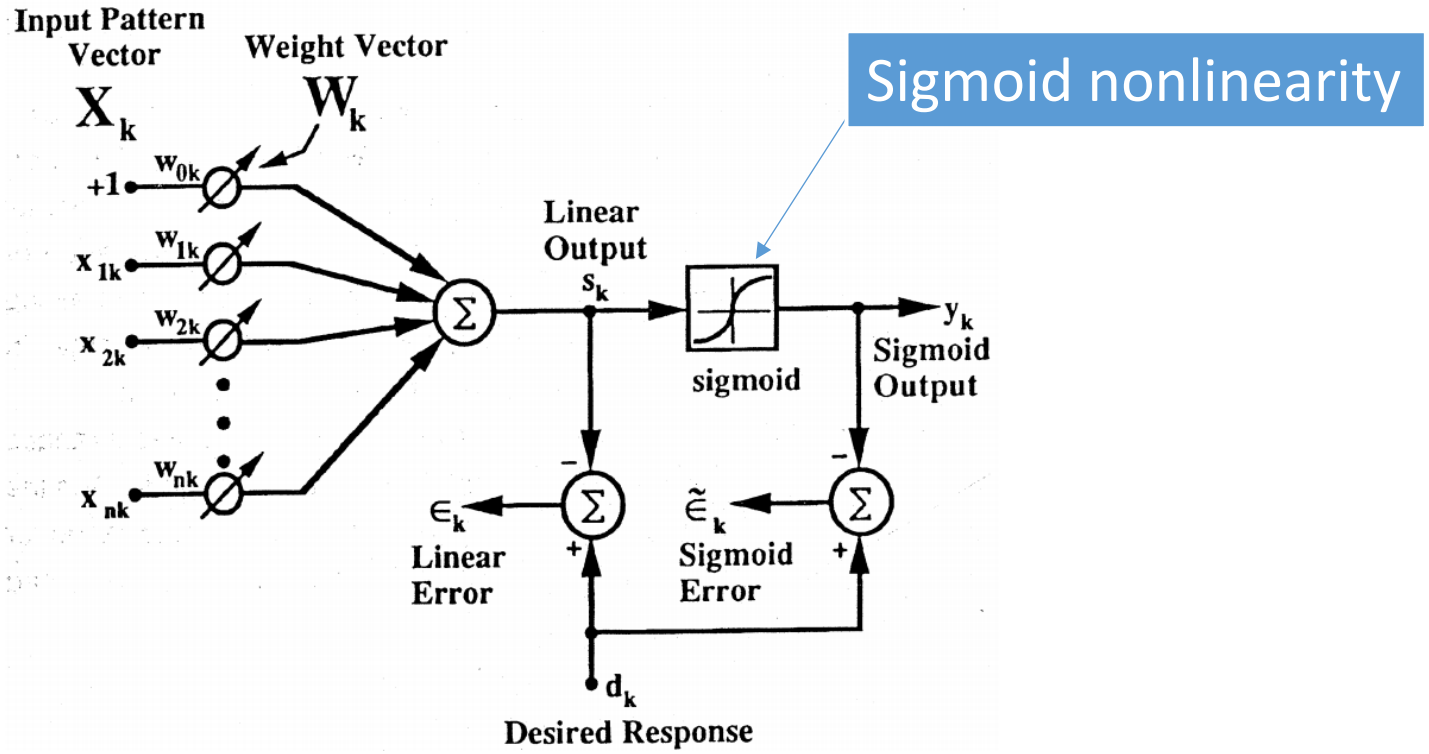
Steepest-Descent – Single Element Linear rules: μ-LMS algorithm

- Small μ, and small K
- The total weight change during the presentation of K of training presentations is proportional to

$$
\begin{aligned}
-\sum_{\ell=0}^{K-1} \frac{\partial \epsilon_{k+\ell}^2}{\partial \mathbf{W}_{k+\ell}} &\simeq -K\left(\frac{1}{K}\sum_{\ell=0}^{K-1}\frac{\partial \epsilon_{k+\ell}^2}{\partial \mathbf{W}_k}\right) \\
&= -K\frac{\partial}{\partial \mathbf{W}_k}\left(\frac{1}{K}\sum_{\ell=0}^{K-1}\epsilon_{k+\ell}^2\right) \\
&\simeq -K\frac{\partial \xi}{\partial \mathbf{W}_k},
\end{aligned}
$$

- ε is the mean-square-error functions.

Steepest-Descent – Single Element Nonlinear rules



- Minimize the mean square of the sigmoid error ε_k:

$$\tilde{\epsilon}_k \triangleq d_k - y_k = d_k - sgm(s_k)$$

Steepest-Descent – Single Element Backpropagation: Sigmoid Adaline

- Instantaneous gradient estimation

$$\hat{\nabla}_k = \frac{\partial(\tilde{\epsilon}_k)^2}{\partial \mathbf{W}_k} = 2\tilde{\epsilon}_k \frac{\partial \tilde{\epsilon}_k}{\partial \mathbf{W}_k}$$

$$\frac{\partial \tilde{\epsilon}_k}{\partial \mathbf{W}_k} = -\frac{\partial sgm(s_k)}{\partial \mathbf{W}_k} = -sgm'(s_k)\frac{\partial s_k}{\partial \mathbf{W}_k}$$

$$s_k = \mathbf{X}_k^T \mathbf{W}_k \qquad \frac{\partial s_k}{\partial \mathbf{W}_k} = \mathbf{X}_k$$

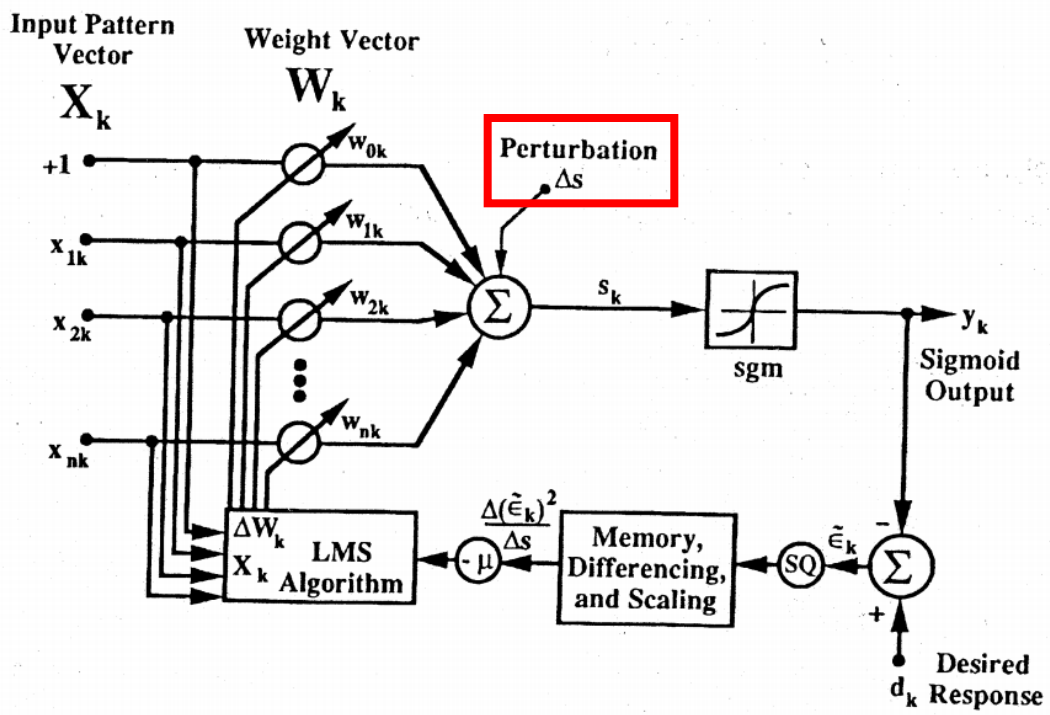
$$\frac{\partial \tilde{\epsilon}_k}{\partial \mathbf{W}_k} = -sgm'(s_k)\mathbf{X}_k \quad \Rightarrow \quad \hat{\nabla}_k = -2\tilde{\epsilon}_k sgm'(s_k)\mathbf{X}_k$$

$$\mathbf{W}_{k+1} = \mathbf{W}_k + \mu(-\hat{\nabla}_k) = \mathbf{W}_k + 2\mu\tilde{\epsilon}_k sgm'(s_k)\mathbf{X}_k$$

Steepest-Descent – Single Element Madaline Rule III: Sigmoid Adaline

- The sigmoid function and its derivative function may not be realized accurately when implemented with analog hardware

- Use a perturbation instead of the derivative of the sigmoid function
- A small perturbation signal Δs is added to the sum s_k
- The effect of this perturbation upon output y_k and error ε_k is noted.



Steepest-Descent – Single Element Madaline Rule III: Sigmoid Adaline

- Instantaneous gradient estimation

$$\hat{\nabla}_k = \frac{\partial(\tilde{\epsilon}_k)^2}{\partial \mathbf{W}_k} = \frac{\partial(\tilde{\epsilon}_k)^2}{\partial s_k}\frac{\partial s_k}{\partial \mathbf{W}_k} = \frac{\partial(\tilde{\epsilon}_k)^2}{\partial s_k}\mathbf{X}_k$$

$$\hat{\nabla}_k \simeq \left(\frac{\Delta(\tilde{\epsilon}_k)^2}{\Delta s}\right)\mathbf{X}_k$$

(Δs is small)

$$\hat{\nabla}_k \simeq 2\tilde{\epsilon}_k\left(\frac{\Delta\tilde{\epsilon}_k}{\Delta s}\right)\mathbf{X}_k$$

- Accordingly

$$\mathbf{W}_{k+1} = \mathbf{W}_k - \mu\left(\frac{\Delta(\tilde{\epsilon}_k)^2}{\Delta s}\right)\mathbf{X}_k \quad OR \quad \mathbf{W}_{k+1} = \mathbf{W}_k - 2\mu\tilde{\epsilon}_k\left(\frac{\Delta\tilde{\epsilon}_k}{\Delta s}\right)\mathbf{X}_k$$

Steepest-Descent – Single Element Backpropagation vs Madaline Rule III

- Mathematically equivalent if the perturbation Δs is small

$$\tilde{\epsilon}_k \stackrel{\Delta}{=} d_k - y_k = d_k - sgm(s_k)$$

- Adding the perturbation Δs causes a change in ε_k equal to

$$\Delta\tilde{\epsilon}_k = -\Delta y_k$$

- Mathematically equivalent if the perturbation Δs is small
- Adding the perturbation Δs causes a change in ε_k equal to

$$\mathbf{W}_{k+1} \simeq \mathbf{W}_k + 2\mu\tilde{\epsilon}_k \frac{\partial y_k}{\partial s_k}\mathbf{X}_k$$

$$= \mathbf{W}_k + 2\mu\tilde{\epsilon}_k sgm'(s_k)\mathbf{X}_k$$

$$\mathbf{W}_{k+1} = \mathbf{W}_k + 2\mu\tilde{\epsilon}_k \left(\frac{\Delta y_k}{\Delta s}\right)\mathbf{X}_k$$

(Δs is small)

Steepest-Descent – Multiple Elements Backpropagation for Networks

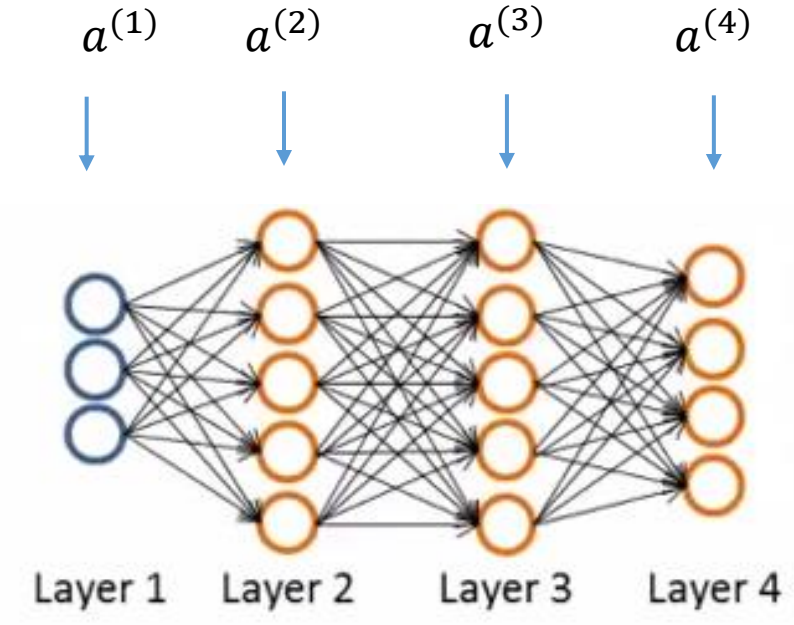
How backpropagation works

- For an input pattern vector X:
  1. Sweep forward through the system to get an output respond vector Y
  2. Compute the errors in each output
  3. Sweep the effects of the errors backward through the network to associate a "square error derivative" δ with each Adaline
  4. Compute a gradient from each δ
  5. Update the weights of each Adaline based upon the corresponding gradient

Steepest-Descent – Multiple Elements Backpropagation for Networks

Forward Propagation

- $a^{(1)} = \boldsymbol{X}$
- $a^{(2)} = sgm((W^{(1)})^T a^{(1)})$
- $a^{(3)} = sgm\left((W^{(2)})^T a^{(2)}\right)$
- $a^{(4)} = sgm((W^{(3)})^T a^{(3)})$



Backpropagation

- For each output unit in layer 4

$$\delta_j^{(4)} = a_j^{(4)} - y_j$$

- $\delta^{(3)} = W^{(3)}\delta^{(4)} .* sgm'(W^{(2)}a^{(2)})$
- $\delta^{(2)} = W^{(2)}\delta^{(3)} .* sgm'(W^{(1)}a^{(1)})$

Intuition: $\delta_j^{(l)}$ = "error" of node j in layer l

Figures are frim Andrew Ng's online course

Steepest-Descent – Multiple Elements Backpropagation for Networks

- Instantaneous gradient for an Adaline element

$$\hat{\nabla}_k = \frac{\partial \epsilon_k^2}{\partial \mathbf{W}_k} = \frac{\partial \epsilon_k^2}{\partial s_k}\frac{\partial s_k}{\partial \mathbf{W}_k}$$

$$\frac{\partial s_k}{\partial \mathbf{W}_k} = \frac{\partial \mathbf{W}_k^T \mathbf{X}_k}{\partial \mathbf{W}_k} = \mathbf{X}_k$$

$$\Rightarrow \quad \hat{\nabla}_k = \frac{\partial \epsilon_k^2}{\partial s_k}\mathbf{X}_k$$

- Define that,

$$\delta_k = -\frac{1}{2}\frac{\partial \epsilon_k^2}{\partial s_k}$$

thus,

$$\hat{\nabla}_k = -2\delta_k \mathbf{X}_k$$

and

$$\mathbf{W}_{k+1} = \mathbf{W}_k + \mu(-\hat{\nabla}_k) = \mathbf{W}_k + 2\mu\delta_k \mathbf{X}_k$$

Steepest-Descent – Multiple Elements Madaline Rule III for Networks

- The sum squared output response error

$$\varepsilon^2 = (d_1 - y_1)^2 + (d_2 - y_2)^2 = \varepsilon_1^2 + \varepsilon_2^2$$

- Provide a perturbation by adding Δs to a selected Adaline, the perturbation propagates through the network, causing the change in the sum of the squares of the errors

$$\Delta(\varepsilon^2) = \Delta(\varepsilon_1^2 + \varepsilon_2^2)$$

$$\frac{\Delta(\varepsilon^2)}{\Delta s} = \frac{\Delta(\epsilon_1^2 + \epsilon_2^2)}{\Delta s} \simeq \frac{\partial(\varepsilon^2)}{\partial s}$$

Steepest-Descent – Multiple Elements Madaline Rule III for Networks

- Instantaneous gradient for an Adaline element

$$\hat{\nabla}_k = \frac{\partial(\epsilon_k^2)}{\partial \mathbf{W}_k} = \frac{\partial(\epsilon_k^2)}{\partial s_k}\frac{\partial s_k}{\partial \mathbf{W}_k} = \frac{\partial(\epsilon_k^2)}{\partial s_k}\mathbf{X}_k$$

- Since

$$\frac{\Delta(\epsilon^2)}{\Delta s} = \frac{\Delta(\epsilon_1^2 + \epsilon_2^2)}{\Delta s} \simeq \frac{\partial(\epsilon^2)}{\partial s}$$

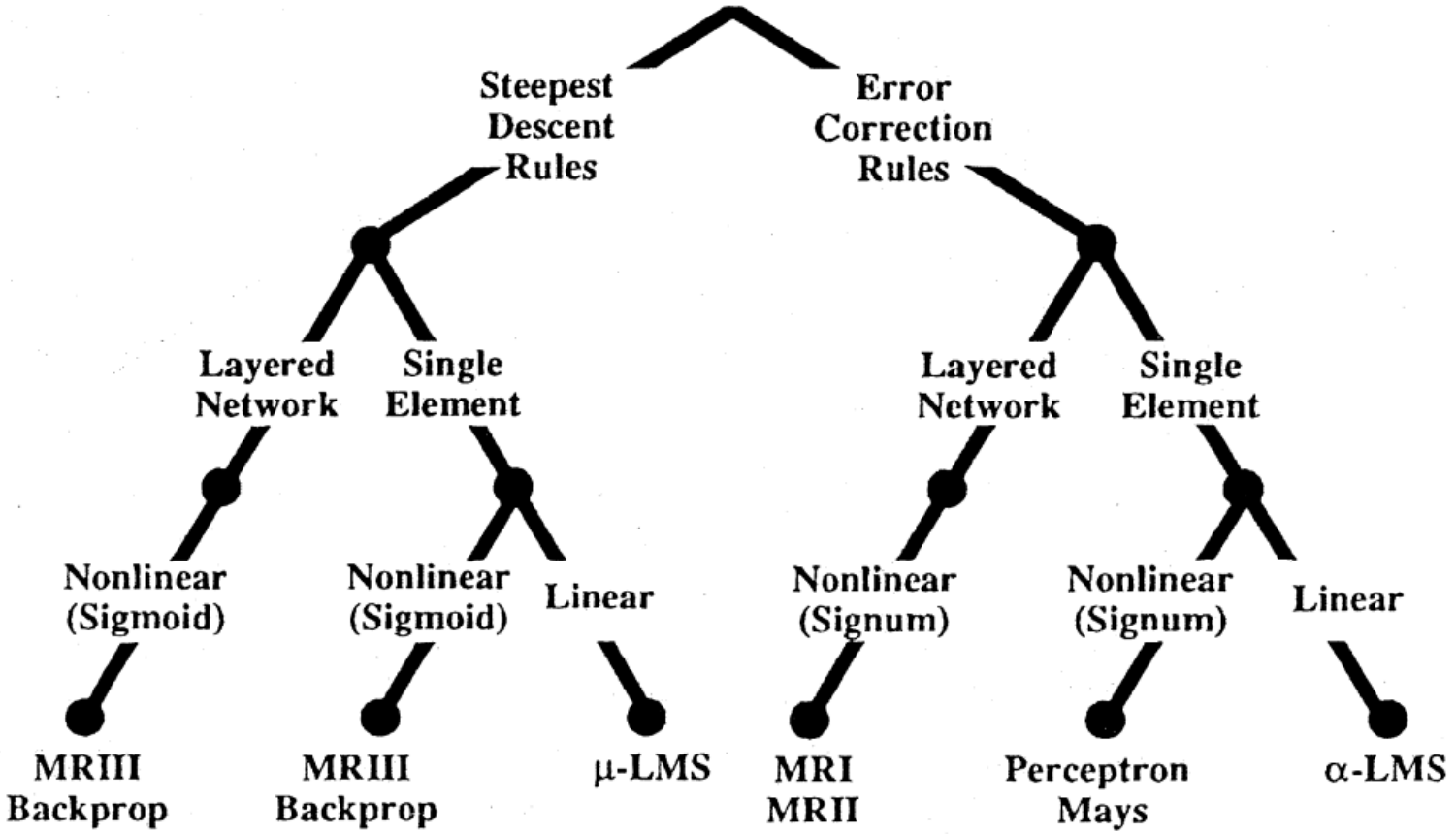
We can get

$$\hat{\nabla}_k \simeq \frac{\Delta(\epsilon_k^2)}{\Delta s}\mathbf{X}_k$$

- Update weight:

$$\mathbf{W}_{k+1} = \mathbf{W}_k - \mu\frac{\Delta(\epsilon_k^2)}{\Delta s}\mathbf{X}_k$$

Steepest-Descent – Multiple Elements Madaline Rule III for Networks



Reference

- B. Widrow and M.E.Hoff, Jr. Adaptive switching circuits. Technical Report 1553-1, Stanford Electron. Labs., Stanford, CA, June 30 1960
- K. Senne. Adaptive Linear Discrete-time estimation. Phd thesis, Technical Report SEL-68-090, Standford Electron. Labs., Stanford, CA, June 1968
- S. Grossberg. Nonlinear neural networks: principles, mechanisms, and architectures. *Neural networks* 1.1 (1988): 17-61.
- S. Cross, R. Harrison, and R. Kennedy. Introduction to neural networks. *The Lancet* 346.8982 (1995): 1075-1079.
- Andrew NG's machine learning course in courera
- LMS tutorials, http://homepages.cae.wisc.edu/~ece539/resources/tutorials/LMS.pdf